



I. REINFORCEMENT LEARNING ALGORITHM:

a. Overview and Components

The environment is set up to be a 8x8 grid where the trained CatBot chooses to move to catch the cat. The goal is to catch the cat in the shortest amount of steps within the 60 move limit. The bot uses Q-learning which is a reinforcement learning algorithm that learns from different actions. In order to train the bot we use Q-Learning Algorithm supported by an off-policy and terminal state. Added by an epsilon-greedy exploration strategy and a reward shaping reward function, trained for 5000 episodes/takes.

b. Q Table Representation

Once the CatBot learns, these learnings are stored in a Q-table, which stores the values of the states and actions it has taken, high values equals a higher probability for a better result. The states combine the CatBots and Cats row and column. Note that there are 4096 possible states from the 8x8 board. As the bot continues to learn the table that initially started with all values set to 0, changes based on experience.

c. Q-Learning Algorithm

The algorithm process is as follows: observe state, choose action, perform action, observe new state, calculate reward and update table. It follows the mathematical formula of $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. This compares past estimates with new estimates and determines which is better. Part of the learning concepts include

1. Off-Policy; which assumes the next action taken is the best action, even if it is random.
2. Terminal State; which returns a value of 0 when cat is caught and there are no future rewards.

d. Exploration Strategy

Catbot uses an epsilon-greedy strategy meaning it prioritizes exploration, otherwise it chooses highest Q-value. SO during training probability solely decreases from 100% to 1%, so it starts off with exploration and ends with focusing on what's been learned. This is why at the start we see a burn-in exploration where the moves are random. After some takes, train against different cat behaviors resulting in trainer-cat noise which improves generalization.

e. Reward Function

We used reward shaping where in each state/environment gives off different points.

1. 0 ; Starting Values
2. 100 ; When the cat is successfully captures
3. -0.2 ; When CatBot moves closer to the cat
4. -3.5 ; When the CatBot does move any closer to the cat.
5. -2.0 ; When the CatBot moves into a wall



With these penalties and rewards in places, it's easier to catch the bot quicker since it prioritizes movement.

f. Training Process

The CatBot is trained for 5000 episodes, each where the board is reset and random burn-in moves are performed, continuing until the cat is successfully caught or the limit has been reached. The learning rate decreases overtime so that learning is faster at the beginning and stable at the end. The discount factor helps the bot value future rewards so that lesser moves are prioritized. Training takes 3 seconds per cat and after everything, the q-table is checked to keep the best strategies.

II. EVALUATION AND PERFORMANCE

To evaluate CatBot's performance after training, we tested it against the five known cats it had been trained to catch. Each test consisted of 100 episodes, with the bot relying only on its learned Q-table and no exploration. Every episode was limited to a maximum of 60 steps. The following results summarize what we observed during testing.

Cat	Success Rate	Avg. Steps to Catch	Fastest	Slowest
Batmeow	100/100 (100%)	14.00	14	14
Mittens	100/100 (100%)	11.79	8	20
Paotsin	100/100 (100%)	15.28	15	22
Peekaboo	95/100 (95.00%)	28.51	21	53
Squiddyboi	100/100 (100%)	20.56	8	51
Average	99.00%	18.03	—	—

```
#####
# TRAINING AGAINST: batmeow
#####
Successfully loaded agent sprite: images/catbot.png
Successfully loaded agent sprite: images/catbot.png

=====
TrainerCat locked in as: Manolo (Matador Cat)
=====

Training complete! Total steps taken: 134170
Average steps per episode: 26.83
Greedy loops repaired: 2177
Training time for batmeow: 2.76s
Successfully loaded agent sprite: images/catbot.png

Evaluating trained bot over 100 test episodes...

=====
EVALUATION BENCHMARK
=====
Success Rate : 100/100 (100.0%)
Avg Catch   : 14.00 steps
Fastest     : 14 steps
Slowest     : 14 steps
=====
```

Figure 1. Sample terminal output training and evaluation for “Batmeow”

```
#####
# TRAINING AGAINST: mittens
#####
Successfully loaded agent sprite: images/catbot.png
Successfully loaded agent sprite: images/catbot.png
Training complete! Total steps taken: 108774
Average steps per episode: 21.75
Greedy loops repaired: 177
Training time for mittens: 2.55s
Successfully loaded agent sprite: images/catbot.png

Evaluating trained bot over 100 test episodes...

=====
EVALUATION BENCHMARK
=====
Success Rate : 100/100 (100.0%)
Avg Catch   : 11.79 steps
Fastest     : 8 steps
Slowest     : 20 steps
=====
```

Figure 2. Sample terminal output training and evaluation for “Mittens”



```
#####
# TRAINING AGAINST: paotsin
#####
Successfully loaded agent sprite: images/catbot.png
Successfully loaded agent sprite: images/catbot.png
Training complete! Total steps taken: 160440
Average steps per episode: 32.09
Greedy loops repaired: 211
Training time for paotsin: 3.83s
Successfully loaded agent sprite: images/catbot.png

Evaluating trained bot over 100 test episodes...

=====
EVALUATION BENCHMARK
=====
Success Rate : 100/100 (100.0%)
Avg Catch   : 15.28 steps
Fastest     : 15 steps
Slowest     : 22 steps
=====
```

Figure 3. Sample terminal output training and evaluation for “Paotsin”

```
#####
# TRAINING AGAINST: peekaboo
#####
Successfully loaded agent sprite: images/catbot.png
Successfully loaded agent sprite: images/catbot.png
Training complete! Total steps taken: 233186
Average steps per episode: 46.64
Greedy loops repaired: 1295
Training time for peekaboo: 4.11s
Successfully loaded agent sprite: images/catbot.png

Evaluating trained bot over 100 test episodes...

=====
EVALUATION BENCHMARK
=====
Success Rate : 95/100 (95.0%)
Avg Catch   : 28.51 steps
Fastest     : 21 steps
Slowest     : 53 steps
=====
```

Figure 4. Sample terminal output training and evaluation for “Peekaboo”

```
#####
# TRAINING AGAINST: squiddyboi
#####
Successfully loaded agent sprite: images/catbot.png
Successfully loaded agent sprite: images/catbot.png
Training complete! Total steps taken: 193313
Average steps per episode: 38.66
Greedy loops repaired: 111
Training time for squiddyboi: 4.19s
Successfully loaded agent sprite: images/catbot.png

Evaluating trained bot over 100 test episodes...

=====
EVALUATION BENCHMARK
=====
Success Rate : 100/100 (100.0%)
Avg Catch   : 20.56 steps
Fastest     : 8 steps
Slowest     : 51 steps
=====
```

Figure 5. Sample terminal output training and evaluation for “Squiddyboi”

Our catbot achieved a 99.00% catch rate against every known cat. After testing 500 test episodes per cat, it never reached the 60-step limit after training. The steps needed to catch each cat depended more on the cat's behavior than the catbot's ability. In summary, Batmeow was always caught the fastest, as it's stationary, taking exactly 14 steps every time. Peekaboo took the longest on average since it teleports away when CatBot gets adjacent. Depending on where it reappeared, it could end up right next to the bot or on the opposite side of the board.

III. CHALLENGES

One of the project's challenges is adapting to unknown/hidden cat behaviours. Since there are 5 cats that are hidden or unknown during the training development, we are not sure if our implemented reward structure and hyperparameters are tailored to the hidden cats' personalities or behavioral patterns. The best solution we found for this is that we implemented another bot for testing, which was made to



De La Salle University – Manila
College of Computer Studies – Computer Technology
CSINTSY MCO3 Submission - Group 101

learn from the personalities and behaviors of the other known cats, so that it can be prepared for and familiar with the hidden cats' behaviors. However, it is not 100% certain that it can match the hidden cats' patterns.

Another challenge is dealing with cats that react to CatBot's own moves. Cats like Paotsin who runs away when chased or Squiddyboi who jumps over the bot when threatened change their behavior based on what CatBot does, basically it is a push and pull relationship so their behavior doesn't stay the same or is predictable, thus the best action to take can depend on what happened before and not fully rely on just the current position. Another challenge is discontinuous transitions. Cats, especially Peekaboo, whose behavior often involves teleporting, break the pattern on distance-based rewards. This breaks the pattern/ logic behind it because "closer" and "farther" no longer make sense, considering the cat's unpredictable position behavior where in some instances it doesn't even land a neighboring cell.

Moreover, another challenge is limited training. Given the fixed 60 moves and the 5000 episodes, this may limit the bot's capacity to learn on how to capture more active cats. And last but not the least, is the limited training budget. With 4096 possible states from the 8x8 grid and 5 actions, the Q-table is too large to cover 5000 training episodes within a 20-second runtime capacity, so full coverage of all state-action pairs isn't guaranteed, especially for less-visited regions like corners and edges.

IV. MEMBER CONTRIBUTION

Parado, Sky Hannah G.	Core Q-learning algorithm design, Report writing and testing
Mojica, Maurienne Marie M.	State-based reward algorithm design, Report writing
Villorente, Khyle Raeka B.	Hyperparameter tuning & Trainer Cat resting, Creating custom_kitties.py and Video editing
Borbe, Ken Marthin H.	Stress testing and reward shaping adjustment, Creating custom_kitties.py, Report writing and testing